до 11 янв., 01:30

Открыть урок

Когда: 14.10.26 в 16:00

Условный оператор и не только — как заставить код принимать решения

- Условный оператор
- Логические операторы
- Вложенные условия
- Подстроки и спецсимволы
- Оператор выбора match-case

Пока наши программы были прямые, как отрезок в геометрии и не предусматривали никакой вариативности. Понятно, что такие программы почти всегда не очень полезны и их надо усложнять. Сегодня наш код немного поумнеет и станет гибким, как йог. Вы узнаете, как управлять потоком программ, используя условия, логику и магию строк. Готовы? Погнали!

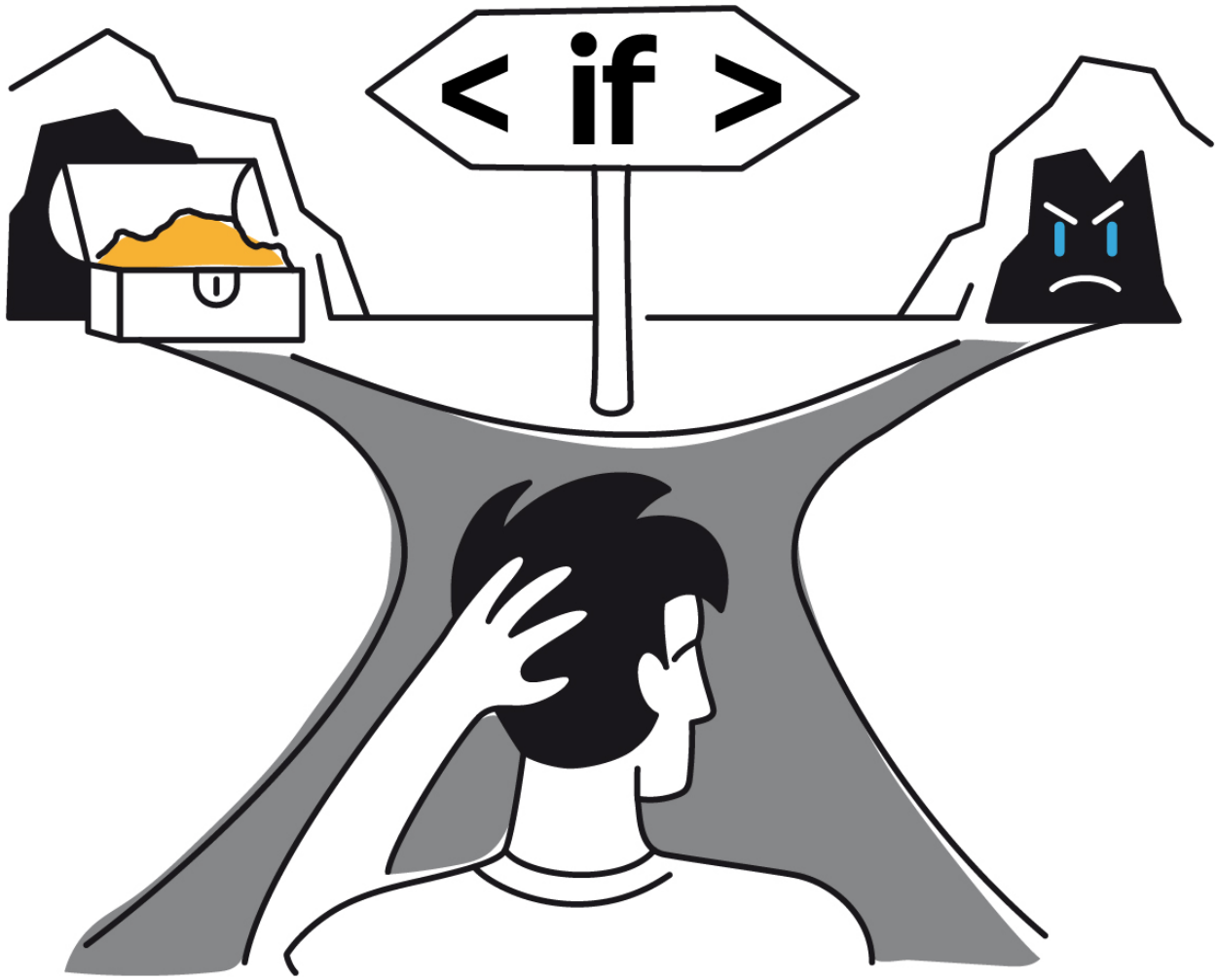
🌲 Условный оператор. Выбор как в RPG

Представьте, что вы — древнегреческий оракул. К вам приходят путники с вопросами:

— О, великий Python, даст ли Зевс дождь завтра?

— Зависит от того, введёшь ли ты правильный пароль, смертный.

Именно так работают условия — они превращают вашу программу из тупого исполнителя команд в подобие разумного существа. Ну почти. В Python для этого есть специальный оператор условия `if-else`.



Представьте, что `if-else` — это развилка в квесте:

— Если пойдёте налево — найдёте сундук с золотом.

— Иначе — нарвётесь на тролля.

```
weather = input()
if weather == "дождь":  # Если пользователь введёт слово "дождь",
    print("Бери зонт и резиновых уточек! 🐥")  # то выполнится этот блок
else:  # Во всех остальных случаях
    print("Солнцезащитные очки — твой выбор!")  # этот блок кода
print("Хорошего дня!")  # Этот код выполнится в любом случае
```

Правила такие: сначала идёт слово `if`, затем условие, потом **обязательно двоеточие** и с новой строки **с отступом в четыре пробела** код, который должен выполниться в том случае, если условие верное.

Часть с `else` **необязательна**, но если она есть, то после `else` **не идёт никакого условия**, а только двоеточие (без пробела) и с новой строки код с отступом в четыре пробела, который должен выполниться, если условие неверное.

> И `if`, и `else` относятся к блочным операторам, для любого из которых в Python верно правило — строчка с оператором обязательно заканчивается двоеточием, затем с новой строки идёт код с отступом в четыре пробела. После блочного оператора должна быть минимум одна строка вложенного кода, но может быть сколько угодно, в том числе другие блочные операторы.

Важно: если `if` может быть без `else`-части, то вот else не может существовать без своего `if`. Еда может существовать без котика, а вот котик без еды — нет!

🔘 Чёрно-белый мир булевой логики

Вы уже обратили внимание, что в предыдущем примере переменную со значением пользовательского ввода мы **сравнили** со строкой с помощью двух знаков равенства. Это один из шести встроенных в Python **операторов отношений**. Вот они все:

- `==` — равно (потому что один знак `=` — оператор присваивания)
- `!=` — не равно
- `>` — больше
- `>=` — больше или равно (пишется как читается)
- `<` — меньше
- `<=` — меньше или равно

> **PEP8**
> Вокруг операторов отношений обязательно нужно ставить по пробелу с каждой стороны.

В результате любой из операций отношения может получиться только два результата — верно (или истинно) или неверно (ложно). В Python для хранения результатов сравнения (и ещё для хранения флагов, которые могут принимать только два состояния — ВКЛ/ВЫКЛ) используется тип данных `bool`. У переменных этого типа только два значения:

- False — «Нет, это был кот!».

```
print(2 == 2)  # True
print(3 < 1)  # False
print(5 <= 13)  # True
print("Привет" == 'привет')  # False — регистр букв имеет значение!
```

Важной особенностью Python является то, что он поддерживает **двойные операции отношения**:

```
x_coord = int(input())
if 0 <= x_coord < 100:
    print('Координата в допустимом диапазоне')
```

Значения типа `bool` можно задавать и напрямую из кода. И **не нужно никаких кавычек**, как для строк:

```
is_pizza_gone = True  # Истинно всё съели
pizza_gone = 'True'  # Строка True
print(is_pizza_gone == pizza_gone)  # Конечно же, значения не равны — Fal
```

Булевы переменные очень удобно использовать в `if-else`:

```
is_cat_awake = True  # Кот не спит → можно гладить
if is_cat_awake:
    print("😸 Мур-мур!")
else:
    print("💤 Кот в отключке.")
```

В Python почти всё можно превратить в `bool`:

- `False`: 0, пустая строка (правильнее сказать — любая пустая коллекция);
- `True`: всё остальное (даже отрицательные числа! и даже 0.000000001!).

Это можно сделать явно с помощью функции `bool()`:

```
print(bool(0))  # False
print(bool(-3.14))  # True
```

Но похожее приведение происходит неявно в операторе `if` — значение оригинальной переменной не меняется, но воспринимается программой как `True`/`False`:

```
print("Как зовут вашего котика?")
pet_name = input()
print("А сколько ему лет?")
pet_age = int(input())

if pet_name:  # Пользователь ввёл любое имя
    print(f"Какое красивое имя — {pet_name}")
else:  # Пользователь просто нажал Enter, и в pet_name — пустая строка
    print("Безымянный котик — тоже котик")

if pet_age:  # Любое ненулевое число
    print("Уже большой!")
else:  # Котику 0 лет
    print('Какой малыш!')
```

🎁 Мы добавили вам условие в условие

Довольно часто бывает необходимость внутри одного условия проверить ещё одно условие. Это легко можно сделать, так как внутри любой части оператора `if-else` может быть другой оператор `if-else`, а внутри второго — третий и так далее. Количество вложенности матрёшки условий ограничено только вашей фантазией и терпением.

```
age = 25
has_money = True

if age >= 18:
    if has_money:
        print("Можно отдохнуть.")
    else:
        print("Ищи работу! 💼")
else:
    print("Иди учи уроки!")
```

Каждый вложенный блок внутри вложенного блока прибавляет ещё 4 пробела к отступу. Как несложно заметить, если сильно увлечься вложенностью, код буквально будет убегать за пределы видимости. Чтобы это не происходило или происходило реже, можно использовать оператор `elif`.

```
age = int(input("Сколько лет тебе, путник?"))

if age >= 3000:
    print("О, великий древний! Расскажи про Python 0.1!")
elif age >= 100:
    print("Добро пожаловать в музей COBOL.")
else:
    print("Юнец, иди учить PEP8!")
```

После `if` может идти сколько угодно операторов `elif`, условия будут проверяться сверху вниз, и проверка прекратится, как только одно из условий будет истинным.

🧩 Логическое трио: not, and, or. Следите за руками (и условиями)

Другим способом избежать превращения кода в ночной кошмар из вложенных условий являются логические операции, которые позволяют делать сложные условия. Их всего три. Вот они в порядке уменьшения приоритета:

- `not` — «не». Инвертирует истину в ложь и наоборот. Срабатывает всегда первым.

```
if not is_cat_hungry:
    print("Кот сыт. Все живы!")
```

- `and` — «и». Чтобы получить истину, надо, чтобы обе части были истинны, иначе — False.

```
if name == "Цезарь" and password == "Ave123":
    print("Добро пожаловать в императорский чат.")
```

- `or` — «или». True получится, даже если хотя бы одно из условий будет истинным.

```
if is_weekend or is_holiday:
    print("Ура, можно валяться в гамаке!")
```

Чтобы повысить приоритет операций, нужно использовать скобки, точно так же, как и в математике. Несколько примеров.

```
print("Идёт дождь?")
is_rain = input() == 'да'
print("А холодно?")
is_cold = input() == 'да'

if is_rain and not is_cold:  # Дождь и не холодно
    print("Зонт и шлёпанцы — ваш выбор! (Дождь, но тепло!)")
elif is_rain and is_cold:  # И дождь, и холодно
    print("Куртка, зонт и горячий чай в термосе!")
else:
    print("Панама и уверенность в себе!")
```

У оператора присваивания самый низкий приоритет, поэтому сначала выполнится `input()`, потом пользовательский ввод сравнится со строкой «да» и только потом результат сравнения `True|False` запишется в переменную.

Иногда нужно посмотреть на условие немного под другим углом, и код сразу станет проще и понятнее. Например, нам надо ввести чётное число, которое больше 3000 и при этом ещё делится на пять или три. И давать пользователю подсказки, если что-то в числе не так. Можно написать что-то вот такое:

```
num = int(input())
if num % 2 == 0 and num > 3000 and (num % 3 == 0 or num % 5 == 0):
    print("Ура!")
    print(f"{num} — подходит")
elif num % 2 != 0:
    print("Нечётное!")
elif num <= 3000:
    print("Меньше или равно 3000")
elif not (num % 3 == 0 or num % 5 == 0):
    print("Не делится ни на три, ни на пять")
```

```python
num = int(input())
if num % 2 != 0:
    print("Нечётное!")
elif num <= 3000:
    print("Меньше или равно 3000")
elif not (num % 3 == 0 or num % 5 == 0):
    print("Не делится ни на три, ни на пять")
else:
    print(f"Ура, {num} — подходит")
```

Смекалочка помогает делать код лучше.

🍉 Вот это понаставили условий

Иногда условий становится настолько много, что они перестают влезать в длину строки, отмеренную нам PEP8, а что ещё хуже, читать их становится просто невозможно.



В таких случаях просто необходимо разбить условие на несколько строчек. Для этого используются круглые скобки:

```
if (user_age > 12 and
        user_balance >= 1000 and
        not is_banned):
    print("Доступ разрешён!")
```

Кроме того, можно записать результат выполнения условия или его части в переменную ещё до оператора `if` для того, чтобы сократить запись и сделать код более читаемым. Сравните:

```
brother2 = input()

if (brother1 == 'Ромул' and brother2 == 'Рем' or
        brother1 == 'Кастор' and (brother2 == 'Поллукс' or brother2 == 'Г
        brother1 == 'Тор' and brother2 == 'Локи' or
        brother1 == 'Марио' and brother2 == 'Луиджи'):
    print('✅ Ага, знаток! Даже поп-культуру захватил!')
else:
    print('❌ Неа. Может, вы перепутали Кастора с Человеком-пауком?')
```

И вариант с записью частей условий в переменную:

```
print('Введите имена двух братьев из мифов, кино или видеоигр:')
brother1 = input()
brother2 = input()

is_roman = brother1 == 'Ромул' and brother2 == 'Рем'
is_greek = brother1 == 'Кастор' and (brother2 == 'Поллукс' or brother2 ==
is_scandinavian = brother1 == 'Тор' and brother2 == 'Локи'
is_marioworld = brother1 == 'Марио' and brother2 == 'Луиджи'

if is_roman or is_greek or is_scandinavian or is_marioworld:
    print('✅ Ага, знаток! Даже поп-культуру захватил!')
else:
    print('❌ Неа. Может, вы перепутали Кастора с Человеком-пауком?')
```

Второй вариант намного проще читать.

💤 Когда ничего не хочется прямо сейчас

Мы уже говорили, что внутри блочного оператора должна быть как минимум одна строчка кода. Но бывает, что мы знаем условие, а вот обработку того, что будет происходить при его истинности, пока не придумали (или нас накрыла внезапная волна лени). В этом случае можно воспользоваться специальной командой-заглушкой `pass`, которая ничего не делает, но при этом наш код не будет падать с синтаксической ошибкой.

```
if user_status == "admin":
```

```
else:
    print("Доступ запрещён! 🛑")
```

Иногда в качестве заглушки используется оператор, который называется `ellipsis`, а записывается как троеточие `...`. Это тоже валидно.

```
if conquer_the_wold:
    # Потом. Сегодня лень.
    ...
```

В некоторых командах разработки договариваются использовать и то и то, но вкладывать в `pass` и `...` немного разный смысл:

- `pass` — заглушка, но так и должно быть (такое бывает)
- `...` — заглушка, но потом тут должен появиться осмысленный код

🔍 Подстрока есть? А если найду?

В Python можно легко проверить, является ли какая-нибудь строка подстрокой другой. Для этого используется оператор `in`.

Оператор `in` нам ещё пригодится, когда мы будем говорить про другие коллекции.

```
text = input()
if "кот-учёный" in text:
    print("Найден кот-ботаник!")
```

Ещё пример:

```
password = input()

if len(password) >= 8 and ("!" in password or "#" in password):
    print("Надёжный пароль! 🔒")
else:
    print("Слабовато...")
```

```
text = input()
if "дракон" not in text:
    print("Драконов нет. Только подземелья.")
```

Хотя синтаксически не будет ошибкой написать `not ("дракон" in text)`, в приличном обществе так никто не делает. И мы, конечно, не будем. Как минимум потому, что по-русски мы говорим не "не подстрока в строке", а "подстрока не в строке".

🖨️ Спецсимволы, или Наследие печатных машинок

Строки в Python (и других языках программирования) могут содержать управляющие последовательности, которые начинаются со знака `\`. Вот самые важные из них:

- `\n` — перенос строки;
- `\r` — возврат каретки (устаревшее, но до сих пор встречается);
- `\t` — табуляция.

```
print("Первая строка\nВторая строка\tТабуляция")
```

В современных операционных системах `\n` объединяет в себе функцию переноса строки и возврата каретки, а раньше строчки в тексте заканчивались комбинацией `\n\r`, что имитировало работу на печатной машинке: `\n` переносил курсор на следующую строчку, а `\r` возвращал его на начало строки.

Кроме того, `\` используется для экранирования символов, например самого себя `\\` или двойной или одинарной кавычки внутри двойных или одинарных кавычек соответственно.

```
print("Хочу пиццу \"Маргариту\"")
print("Путь: C:\\Users\\Кот")
```

✨ Выбираем со стилем

Во многих языках программирования есть оператор выбора `switch`. В Python есть его

величию и будем использовать его для замены `if-elif-....-elif-else` в случаях, когда проверяется одна переменная, но у неё может быть много значений:

```
day = input()
match day:  # Смотрим, какое значение у day
    case "понедельник":  # if day == "понедельник"
        print("Начало рабочей недели...")
    case "суббота" | "воскресенье":  # if day == "суббота" or day =="воск
        print("Уиии! Выходные! 🎉")
    case _:  # else
        print("Любой другой день.")
```

Фишки:

- `|` — логическое ИЛИ;
- `_` — дефолтный кейс (как `else`).

```
pizza_pieces = int(input())
match pizza_pieces:
    case 0:
        print("Мне не хватило(")
    case 1:
        print("Спасибо, что оставили, но мало.")
    case 2 | 3 | 4:
        print("Это по мне!")
        print("Ура!")
    case _:
        print("Съем во что бы то ни стало!")
```

Разумеется, каждый вложенный блок в `case` может содержать любое количество строчек кода.

🌟 Итог: наш код проберётся через самый сложный перекрёсток

Мы узнали:

- как комбинировать условия;

- что нужно сделать, чтобы не потеряться в многоэтажных условиях;
- как найти кота в любой строке (или не найти).

P. S. Не оставляй `pass` на завтра в коде, который можно написать сегодня.